Creating a Database Table in Magento 2

Declarative schema, `db_schema.xml`, and everything you built on top of `codilar_twoxl_product_enquiry`

Project: 2XL Home · Module: `Codilar_Twoxl`
Table: `codilar_twoxl_product_enquiry`
Generated 30 October 2026

What is inside this guide

1. **Part 1 — Why `db_schema.xml` exists.** The old way, the new way, and why Magento changed.
2. **Part 2 — Your file, line by line.** Every attribute in your schema explained.
3. **Part 3 — Column types.** The complete list, and why you picked each one.
4. **Part 4 — Constraints.** Primary, foreign, unique — and the `onDelete` decision that matters most.
5. **Part 5 — Indexes.** What they really do, and why your table has exactly four.
6. **Part 6 — `db_schema_whitelist.json`.** The file everyone forgets, and what breaks when you do.
7. **Part 7 — Changing a table later.** Add, rename, drop, and how rollback works.
8. **Part 8 — The commands.** `setup:upgrade`, dry-run, status, whitelist generation.
9. **Part 9 — From table to feature.** Model, ResourceModel, Collection, Factory — the four classes and what each is for.
10. **Part 10 — The admin UI.** di.xml, grid, form, DataProvider, controllers, ACL, menu, buttons.
11. **Part 11 — The storefront write path.** How a row actually gets created.
12. **Part 12 — Traps in your own code.** Five real bugs and their fixes.
13. **Part 13 — Cheat sheet, commands, SQL and exercises.**

Part 1 — Why `db_schema.xml` exists

To understand declarative schema you need to know what it replaced — because the pain of the old way *is* the reason the new way looks like it does.

The old way (Magento 2.0 – 2.2): scripts

You wrote PHP that executed DDL by hand:

```
// InstallSchema.php — ran once, on first install
$table = $setup->getConnection()->newTable('codilar_twoxl_product_enquiry')
    ->addColumn('entity_id', Table::TYPE_INTEGER, null, ['identity' => true, 'primary' =>
true])
    ->addColumn('name', Table::TYPE_TEXT, 120, ['nullable' => false]);
$setup->getConnection()->createTable($table);

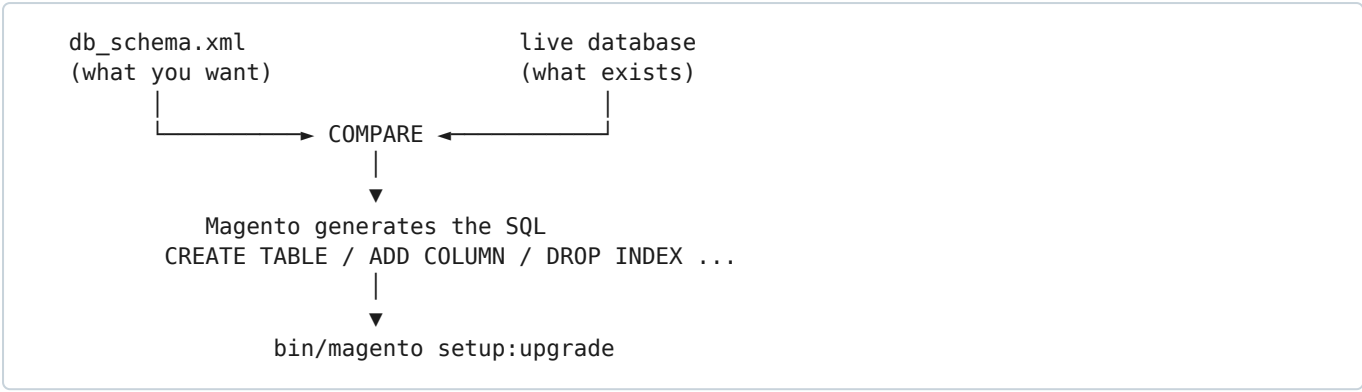
// UpgradeSchema.php — ran on every version bump, full of if-statements
if (version_compare($context->getVersion(), '1.0.1', '<')) {
    $setup->getConnection()->addColumn($table, 'phone', [...]);
}
if (version_compare($context->getVersion(), '1.0.2', '<')) {
    $setup->getConnection()->addColumn($table, 'store_id', [...]);
}
```

Four problems made this miserable in real projects:

1. **You described the journey, not the destination.** To know what a table looked like you had to read the install script *and* mentally replay every upgrade step in order.
2. **No rollback.** Downgrading a module left the database in whatever state the last upgrade put it. You wrote the "undo" by hand or lived with it.
3. **Version tracking broke constantly.** Everything hinged on `setup_module.schema_version` matching `module.xml`. One mismatched number and upgrades silently skipped.
4. **Merge conflicts.** Two developers adding columns in the same sprint collided in the same if-chain.

The new way (2.3+): describe the end state

> **The single idea behind declarative schema:** you write down what the table should **look like**, not the steps to get there. Magento compares your XML to the live database and works out the `ALTER` statements itself.



Benefits you get for free:

- **The file *is* the documentation.** One look tells you the current structure.
- **Rollback works.** Revert the module to an older version, run `setup:upgrade`, and Magento generates the reverse changes.
- **No version numbers to manage.** Notice that your `etc/module.xml` has **no** `setup_version` attribute at all — that is correct for 2.3+ and only possible because of declarative schema.
- **Dry-run.** You can see the SQL before it runs.

What declarative schema does NOT do

> **Schema vs data — keep these separate in your head.**
> `db_schema.xml` handles **structure**: tables, columns, keys, indexes.
> **Data patches** (`Setup/Patch/Data/*.php`) handle **content**: inserting rows, updating values, creating attributes.
>
> Your project shows both: `db_schema.xml` creates the empty enquiry table; `SeedProductEnquiries.php` puts two demo rows in it. Never try to insert data from `db_schema.xml` — there is no syntax for it, by design.

Part 2 — Your file, line by line

`app/code/Codilar/Twoxl/etc/db_schema.xml`

The wrapper

```
<?xml version="1.0"?>
<schema xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

xsi:noNamespaceSchemaLocation="urn:magento:framework:Setup/Declaration/Schema/etc/schema.x
sd">
```

The `xsi:noNamespaceSchemaLocation` is not decoration — it points your IDE at the XSD so you get autocomplete and validation. If PhpStorm shows no suggestions inside `<schema>`, that line is wrong.

The table element

```
<table name="codilar_twoxl_product_enquiry" resource="default" engine="innodb"
       comment="2XL product enquiries raised from the storefront">
```

Attribute	Your value	What it means
`name`	`codilar_twoxl_product_enquiry`	The real MySQL table name. Convention is `vendor_module_entity`, lowercase, snake_case. Yours follows it exactly, which matters because it guarantees no collision with another vendor's module.
`resource`	`default`	Which database connection. Options: `default`, `checkout`, `sales`. Only split these if the client actually runs separate DBs (rare, and Adobe deprecated it). `default` is right.
`engine`	`innodb`	Only `innodb` or `memory` are allowed. **InnoDB is required for foreign keys** — your `store_id` FK would be silently ignored on MyISAM. Always innodb.
`comment`	"2XL product enquiries…"	Stored in MySQL as the table comment. Shows up in `SHOW CREATE TABLE` and in GUI tools. Free documentation — always fill it in.

> **Table name length limit:** MySQL allows 64 characters, but Magento generates index and constraint names from the table name, and *those* can overflow. Keep table names under about 40 characters. Yours is 29 — comfortable.

The columns

```
<column xsi:type="int" name="entity_id" unsigned="true" nullable="false" identity="true"
        comment="Enquiry ID"/>
```

This is the primary key. Four attributes, all deliberate:

- `xsi:type="int"` — the column type. This attribute is mandatory on every column.
- `unsigned="true"` — no negative values, which doubles the usable range. IDs are never negative.
- `nullable="false"` — a primary key can never be null.
- `identity="true"` — this is `AUTO_INCREMENT`. Only one column per table may have it.

```
<column xsi:type="varchar" name="name" length="120" nullable="false" comment="Customer
name"/>
<column xsi:type="varchar" name="email" length="180" nullable="false" comment="Customer
email"/>
<column xsi:type="varchar" name="phone" length="40" nullable="true" comment="Customer
phone"/>
<column xsi:type="varchar" name="sku" length="64" nullable="true"
        comment="SKU the enquiry is about; null for a general enquiry"/>
```

Notice how the lengths were chosen, and how the application respects them.

`app/code/Codilar/Twoxl/Controller/Enquiry/Save.php`:

```
return [
    'name'    => mb_substr($name, 0, 120),     // matches length="120"
    'email'   => mb_substr($email, 0, 180),    // matches length="180"
    'phone'   => $phone   !== '' ? mb_substr($phone, 0, 40)  : null,
    'sku'     => $sku     !== '' ? mb_substr($sku, 0, 64)    : null,
    'message' => $message !== '' ? mb_substr($message, 0, $messageMax) : null,
];
```

> **Why `mb_substr` and not `substr`.** MySQL counts `VARCHAR(120)` in *characters*; PHP's `substr` counts *bytes*. An Arabic or Hindi name is multi-byte, so `substr($name, 0, 120)` would cut mid-character and could still exceed the column. `mb_substr` counts characters, matching MySQL. On a UAE/India project this is not theoretical.

And the lengths themselves follow real standards:

- **email = 180.** RFC 5321 caps an address at 254, but Magento uses 128–255 across core; 180 is safe and keeps the index small. (An index on a `varchar(255)` utf8mb4 column is 1020 bytes and can hit MySQL's 767-byte legacy key limit.)
- **sku = 64.** Matches `catalog_product_entity.sku` exactly — essential, because you compare the two.
- **phone = 40.** Room for `+971 50 123 4567` plus extensions. Note it is `varchar`, never a numeric type — phone numbers have `+`, spaces and leading zeros.

```
<column xsi:type="text" name="message" nullable="true" comment="Enquiry message"/>
```

`text` takes no `length` — it is always 65,535 bytes. Your config caps the message at 2,000 characters by default and hard-clamps it at 5,000 in `EnquiryConfig::getMessageMaxLength()`, which is why the comment in that file reads *"also the column's practical ceiling"*.

```
<column xsi:type="smallint" name="status" unsigned="true" nullable="false"
identity="false"
        default="1" comment="1 new, 2 in progress, 3 closed"/>
```

A status column done properly:

- **`smallint`, not `varchar`.** Storing `'new'` / `'closed'` as text means every filter is a string comparison and a typo becomes a silent bug. An integer with named constants is faster and type-safe.
- **`default="1"`.** A row inserted without a status is automatically "New". Belt and braces — `Save.php` also sets it explicitly.
- **The comment documents the enum** right there in the database, for the DBA who will never read your PHP.
- `identity="false"` is the default; stating it is harmless but unnecessary.

The constants live in `app/code/Codilar/Twoxl/Model/ProductEnquiry.php`:

```
public const STATUS_NEW = 1;
public const STATUS_IN_PROGRESS = 2;
public const STATUS_CLOSED = 3;

public const STATUSES = [
    self::STATUS_NEW => 'New',
    self::STATUS_IN_PROGRESS => 'In Progress',
    self::STATUS_CLOSED => 'Closed',
];
```

> **The pattern to copy:** integer in the database → constants in the model → one `OptionSourceInterface` class that turns the constants into dropdown options. Your `Model/Source/EnquiryStatus.php` is that third piece, and it is used by **both** the grid filter and the edit form. Define the list once; never hard-code status labels in XML.

```
<!-- Nullable so an enquiry outlives the store view it came from. -->
<column xsi:type="smallint" name="store_id" unsigned="true" nullable="true"
identity="false"
        comment="Store view the enquiry was raised in"/>
```

The type and sign here are not a style choice — they are a hard requirement. A foreign key only works if both columns have **identical** definitions, and `store.store_id` is `smallint unsigned`. Use `int` here and `setup:upgrade` fails with a constraint error.

```
<column xsi:type="timestamp" name="created_at" on_update="false" nullable="false"
        default="CURRENT_TIMESTAMP" comment="Created at"/>
<column xsi:type="timestamp" name="updated_at" on_update="true" nullable="false"
        default="CURRENT_TIMESTAMP" comment="Updated at"/>
```

This pair is the Magento idiom for audit timestamps, and the difference is one attribute:

Column	`on_update`	SQL it generates	Behaviour
`created_at`	`false`	`DEFAULT CURRENT_TIMESTAMP`	Set once on INSERT, never changes again.
`updated_at`	`true`	`DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP`	Set on INSERT **and** refreshed on every UPDATE, by MySQL itself.

> **MySQL does this, not PHP.** You never write `setCreatedAt()` anywhere in your module — and indeed you don't. Look at `Controller/Enquiry/Save.php` : it sets name, email, phone, sku, message, status and store_id. The timestamps appear by themselves. That is also why they are correct even if a row is inserted by a SQL script or an import.

Part 3 — Column types: the complete reference

Every type you can write in `xsi:type`, what it maps to in MySQL, and when to reach for it.

Numbers

xsi:type	MySQL	Range (unsigned)	Use for
`tinyint`	TINYINT	0 – 255	Booleans, tiny flags. `is_active`, `is_default`.
`smallint`	SMALLINT	0 – 65,535	Store ids, small enums. **Your `status` and `store_id`.**
`int`	INT	0 – 4.29 billion	Entity ids, foreign keys to big tables. **Your `entity_id`.**
`bigint`	BIGINT	0 – 1.8×10^{19}	Log tables, very high-volume ids. Rarely needed.
`decimal`	DECIMAL(p,s)	exact	**Money. Always.** `precision="12" scale="4"` is the Magento standard for prices.
`float`	FLOAT	approximate	Measurements where rounding error is acceptable. **Never for money.**
`double`	DOUBLE	approximate	Scientific values. Almost never in e-commerce.
`real`	FLOAT	approximate	Legacy alias. Avoid.

Never store money in `float`. `0.1 + 0.2` is not `0.3` in binary floating point. On a cart total that becomes a one-fils discrepancy the client's accountant will find. Magento uses `decimal precision="20" scale="4"` for prices — four decimal places so tax calculations do not lose precision before rounding.

```
<column xsi:type="decimal" name="quoted_price" precision="20" scale="4"
        unsigned="false" nullable="true" default="0" comment="Quoted price"/>
```

Text

xsi:type	MySQL	Max size	Use for
`varchar`	VARCHAR(n)	n ≤ 65,535	Short text you may want to index or search. **Your name, email, phone, sku.** `length` is required.
`char`	CHAR(n)	n ≤ 255	Fixed-length codes only — country codes, hashes. Pads with spaces otherwise.
`text`	TEXT	64 KB	Long free text. **Your `message`.** Cannot have a default value.
`mediumtext`	MEDIUMTEXT	16 MB	Serialised payloads, long HTML, CMS content.
`longtext`	LONGTEXT	4 GB	Rarely justified. Consider a file instead.
`blob` / `mediumblob` / `longblob`	BLOB family	64 KB – 4 GB	Binary data. Usually a mistake — store files on disk and keep the path.

Dates and booleans

xsi:type	MySQL	Notes
`timestamp`	TIMESTAMP	1970–2038, stored UTC and converted by the connection timezone. Supports `CURRENT_TIMESTAMP` defaults and `on_update`. **Your created_at / updated_at.**
`datetime`	DATETIME	1000–9999, no timezone conversion. Use for future dates beyond 2038, or when you want the literal value preserved.
`date`	DATE	Day only, no time.
`boolean`	BIT(1)	Allowed, but Magento core overwhelmingly prefers `smallint` or `tinyint` with 0/1. Follow core.

Attributes available on a column

Attribute	Applies to	Meaning
`name`	all	Column name. Required.
`xsi:type`	all	The type. Required.
`nullable`	all	`true` = NULL allowed. Default is `true`, so state it explicitly either way.
`default`	most	Default value. Not allowed on `text` / `blob`.
`comment`	all	Column comment. Always write one.
`unsigned`	numeric	No negatives, double the positive range.
`identity`	integer	AUTO_INCREMENT. One per table.
`length`	varchar, char	Required for these two.
`precision` / `scale`	decimal	Total digits / digits after the point.
`on_update`	timestamp	`true` adds ON UPDATE CURRENT_TIMESTAMP.
`disabled`	all	`true` deletes the column. See Part 7.
`onCreate`	all	Migration instruction for renames. See Part 7.

Part 4 — Constraints

Constraints are rules the *database* enforces, so they hold even when a row is inserted by a script, an import or a direct SQL statement that never touched your PHP.

Primary key

```
<constraint xsi:type="primary" referenceId="PRIMARY">
    <column name="entity_id"/>
</constraint>
```

The `referenceId` must be the literal string `PRIMARY` — that is what MySQL calls it. Listing more than one `<column>` makes a composite key, which you want for link tables (e.g. `enquiry_id` + `tag_id`) but not here.

Foreign key — the important one

```
<constraint xsi:type="foreign"
            referenceId="CODILAR_TWOXL_PRODUCT_ENQUIRY_STORE_ID_STORE_STORE_ID"
            table="codilar_twoxl_product_enquiry" column="store_id"
            referenceTable="store" referenceColumn="store_id"
            onDelete="SET NULL"/>
```

Attribute	Meaning
`referenceId`	The constraint's name. Magento's convention is `THIS_TABLE_THIS_COLUMN_OTHER_TABLE_OTHER_COLUMN`, uppercased. Yours follows it exactly.
`table` / `column`	This side of the relationship.
`referenceTable` / `referenceColumn`	The other side — here Magento's own `store` table.
`onDelete`	What happens to **your** row when the referenced row is deleted.

The `onDelete` choice

Value	Effect	When to use it
`CASCADE`	Your row is deleted too.	True child data that is meaningless alone — order items when the order goes, a product's images.
`SET NULL`	Your column becomes NULL, the row survives.	**Your choice.** Context data that is nice to have but not essential. Requires `nullable="true"`.
`RESTRICT`	The delete is refused.	When losing the parent must be prevented outright.
`NO ACTION`	Same as RESTRICT in InnoDB.	Effectively an alias.

> **Why `SET NULL` is the right answer here, and `CASCADE` would be a disaster.**
> Imagine the client closes the India store view two years from now. With `CASCADE`, deleting that store view would silently delete **every enquiry ever raised in India** — sales leads, trade quotes, the lot — with no warning and no undo. With `SET NULL` the enquiries remain; they just no longer know which store they came from. Your comment says it in one line: *"Nullable so an enquiry outlives the store view it came from."*
>
> Magento core makes the same call: `sales_order.store_id` is `SET NULL`. Customer-generated records should outlive configuration.

Unique constraint

You do not have one, and that is correct — the same person may legitimately enquire about several products, or twice about one. But here is the syntax for when you need it:

```
<constraint xsi:type="unique" referenceId="CODILAR_TWOXL_PRODUCT_ENQUIRY_EMAIL_SKU">
    <column name="email"/>
    <column name="sku"/>
</constraint>
```

Interestingly, your `SeedProductEnquiries` patch enforces exactly this rule — but in PHP, where it belongs for seed data:

```
private function exists(string $email, string $sku): bool
{
    return (bool)$this->collectionFactory->create()
        ->addFieldToFilter('email', $email)
        ->addFieldToFilter('sku', $sku)
        ->getSize();
}
```

That is what makes the patch **idempotent** — safe to run twice. A database-level unique constraint would have thrown an exception instead and aborted `setup:upgrade`.

Part 5 — Indexes

An index is a sorted lookup structure. Without one, finding rows means reading **every row** in the table. With one, the database jumps straight to the matches.

Think of a 400-page book. Finding every mention of "sofa" without an index means reading all 400 pages. The index at the back lists it once with page numbers. Same idea, same trade-off: the index takes extra space, and it must be updated every time the book changes.

Your four indexes, and the exact query each one serves

```
<index referenceId="CODILAR_TWOXL_PRODUCT_ENQUIRY_STATUS" indexType="btree">
    <column name="status"/>
</index>
<index referenceId="CODILAR_TWOXL_PRODUCT_ENQUIRY_SKU" indexType="btree">
    <column name="sku"/>
</index>
<index referenceId="CODILAR_TWOXL_PRODUCT_ENQUIRY_EMAIL" indexType="btree">
    <column name="email"/>
</index>
<index referenceId="CODILAR_TWOXL_PRODUCT_ENQUIRY_CREATED_AT" indexType="btree">
    <column name="created_at"/>
</index>
```

Index	Serves	Found in
`status`	The Status dropdown filter, and "open enquiries" queries	`Collection::addOpenFilter()` — `addFieldToFilter('status', ['neq' => STATUS_CLOSED])`
`sku`	"All enquiries about this product"	The SKU grid filter; `SeedProductEnquiries::exists()`
`email`	"All enquiries from this person"	The Email grid filter; `SeedProductEnquiries::exists()`
`created_at`	Date-range filtering and default sorting	The `dateRange` filter on the Received column; `setOrder('created_at', DESC)`

The rule that decides whether to add an index: index a column that appears in a `WHERE`, `ORDER BY` or `JOIN` — and only then. Every index costs disk space and slows down every INSERT and UPDATE, because the index has to be maintained too. On a write-heavy table, unnecessary indexes are a real cost.

Your four map one-to-one onto the four filterable/sortable columns in the admin grid. That is not a coincidence — it is the correct way to decide.

What you did NOT index, and why that is right

- `name` — filterable in the grid, but names are rarely searched and the table will stay small. A judgement call; defensible either way.
- `phone` — filterable, but nobody searches enquiries by phone number.
- `message` — a `TEXT` column. A plain BTREE index on TEXT requires a prefix length and is nearly useless; full-text search is a different tool.
- `entity_id` — already indexed, automatically, as the primary key.
- `store_id` — **already indexed automatically by the foreign key.** InnoDB creates an index on any FK column that does not already have one. Adding `<index>` for it would create a duplicate. This is a genuinely easy mistake to make, and your schema avoids it.

Index types

indexType	Use
`btree`	The default and the right answer ~99% of the time. Handles equality, ranges and sorting.
`fulltext`	Word search inside long text. Only on CHAR/VARCHAR/TEXT, and only with MATCH…AGAINST queries.
`hash`	MEMORY engine only. Equality lookups only — no ranges, no sorting. Effectively unusable for InnoDB tables.

Composite indexes — the one thing worth knowing

An index can cover several columns, and **order matters**. An index on `(status, created_at)` helps queries that filter on `status` alone, or on `status` *and then* sort by `created_at`. It does **not** help a query that only touches `created_at`.

```
<index referenceId="CODILAR_TWOXL_PRODUCT_ENQUIRY_STATUS_CREATED_AT" indexType="btree">
    <column name="status"/>
    <column name="created_at"/>
</index>
```

This is the "left-prefix rule": an index on (A, B, C) serves queries on A, on (A,B), and on (A,B,C) — never on B alone. If your enquiry table ever grows to hundreds of thousands of rows, this composite index would be the ideal one for `addOpenFilter()`, which filters status *and* sorts by created_at in a single query.

Part 6 — `db_schema_whitelist.json`

This is the file every Magento developer forgets at least once. Understanding it takes two minutes and saves you an afternoon.

The problem it solves

Your module is not the only thing that touches your table. A third-party extension, a client's custom module, or an old install script might have added a column. If Magento simply made the database match `db_schema.xml`, it would **drop** anything not listed — including other people's work and, in the worst case, live data.

So Magento only ever removes something if it can prove **your module created it**. The whitelist is that proof.

```
  Is this column/index/constraint in db_schema.xml?
      │
      ├── YES → make sure the database matches it
      │
      └── NO  → is it in db_schema_whitelist.json?
                  │
                  ├── YES → this module created it, and the dev removed it
                  │           from the XML on purpose  →  DROP IT
                  │
                  └── NO  → somebody else owns this  →  LEAVE IT ALONE
```

Your file

`app/code/Codilar/Twoxl/etc/db_schema_whitelist.json`

```
{
    "codilar_twoxl_product_enquiry": {
        "column": {
            "entity_id": true, "name": true, "email": true, "phone": true,
            "sku": true, "message": true, "status": true, "store_id": true,
            "created_at": true, "updated_at": true
        },
        "index": {
            "CODILAR_TWOXL_PRODUCT_ENQUIRY_STATUS": true,
            "CODILAR_TWOXL_PRODUCT_ENQUIRY_SKU": true,
            "CODILAR_TWOXL_PRODUCT_ENQUIRY_EMAIL": true,
            "CODILAR_TWOXL_PRODUCT_ENQUIRY_CREATED_AT": true
        },
        "constraint": {
            "PRIMARY": true,
            "CODILAR_TWOXL_PRODUCT_ENQUIRY_STORE_ID_STORE_STORE_ID": true
        }
    }
}
```

Three sections — `column`, `index`, `constraint` — each listing names with `true`. The value is always `true`; only the presence of the key matters.

You never write this file by hand

```
bin/magento setup:db-declaration:generate-whitelist --module-name=Codilar_Twoxl
```

Or for every module at once:

```
bin/magento setup:db-declaration:generate-whitelist
```

The workflow that keeps you out of trouble:

1. Edit `db_schema.xml`.
2. Run `generate-whitelist` for your module. **Immediately. Every time.**
3. Run `setup:upgrade`.
4. Commit **both** files together.

What goes wrong when you skip it. You add a column to `db_schema.xml`, run `setup:upgrade`, and it works fine on your machine — adding is always allowed. Months later you remove that column from the XML. Because it was never whitelisted, Magento does not recognise it as yours and **leaves it in the database**. Now production and the XML disagree, `setup:db:status` reports the database as out of date forever, and the next developer cannot tell which is correct.

The whitelist is append-only in practice: the generator adds entries, and entries stay even after you drop the column. That is deliberate — the record that "we once owned this" is what makes the eventual DROP safe.

It is a generated file, but it belongs in git. It must ship with the module, because the production server needs it to make the same decisions your dev machine made. Never add it to `.gitignore`.

Part 7 — Changing the table later

Your table exists now. Here is how every future change is made.

Adding a column

Just add the element. Magento generates `ALTER TABLE … ADD COLUMN`.

```
<column xsi:type="varchar" name="assigned_to" length="120" nullable="true"
        comment="Admin user handling this enquiry"/>
```

> **Adding a `nullable="false"` column to a table that already has rows will fail** unless you also give it a `default`. Existing rows need *something* to put in the new column. Either make it nullable, or give it a default:
>
> ```
> <column xsi:type="smallint" name="priority" unsigned="true" nullable="false"
>         default="2" comment="1 high, 2 normal, 3 low"/>
> ```

Changing a column

Edit the attribute in place. Widening is safe; narrowing truncates data.

```
<!-- was length="120" -->
<column xsi:type="varchar" name="name" length="180" nullable="false" comment="Customer
name"/>
```

If you change a length, remember to update the matching `mb_substr()` in `Controller/Enquiry/Save.php`. The schema and the validation must agree, or you get silent truncation at one end or a database error at the other.

Dropping a column

Delete the element from `db_schema.xml`, then regenerate the whitelist. The entry stays in the JSON — that is what authorises the DROP.

Renaming a column — the one with a trick

Deleting `phone` and adding `mobile` would drop the column **and all its data**, then create an empty one. To carry the data across, use `onCreate`:

```
<column xsi:type="varchar" name="mobile" length="40" nullable="true"
        onCreate="migrateDataFrom(phone)" comment="Customer mobile"/>
```

Same for a whole table:

```
<table name="codilar_twoxl_enquiry" resource="default" engine="innodb"
       onCreate="migrateDataFromAnotherTable(codilar_twoxl_product_enquiry)">
```

`onCreate` runs **only when the new column or table is created**, which is exactly once. After the migration has run everywhere, the attribute is dead weight — but leaving it is harmless and documents the history.

Dropping a table

```
<table name="codilar_twoxl_product_enquiry" disabled="true"/>
```

Keep the element with `disabled="true"` rather than deleting it. The same attribute works on a single column, index or constraint. This is the declarative way to say "remove this" while leaving a record that you meant to.

Extending someone else's table

This is a daily task on real projects. You declare a `<table>` with an existing core name and list **only your additions** — Magento merges every module's `db_schema.xml` into one picture before comparing with the database.

```
<!-- in YOUR module's db_schema.xml -->
<table name="sales_order">
    <column xsi:type="varchar" name="twoxl_delivery_slot" length="40" nullable="true"
            comment="Chosen delivery slot"/>
</table>
```

You do not repeat `sales_order`'s existing columns, and you must not set `engine` or `resource` on a table you do not own.

> **This merging is the deepest reason declarative schema exists.** In the old script world, two modules extending `sales_order` had to run in the right order and check whether the other had already run. Now each module states its own contribution and Magento resolves the whole picture.

Part 8 — The commands

```
# Apply schema changes AND run data patches
bin/magento setup:upgrade

# Schema only, skip data patches (faster while iterating on the XML)
bin/magento setup:upgrade --keep-generated --safe-mode=1

# Show the SQL that WOULD run, without touching the database
bin/magento setup:upgrade --dry-run=1

# Is the database in sync with the XML?
bin/magento setup:db:status

# Regenerate the whitelist after EVERY schema edit
bin/magento setup:db-declaration:generate-whitelist --module-name=Codilar_Twoxl

# Is the module even enabled?
bin/magento module:status Codilar_Twoxl

# After schema changes
bin/magento cache:flush
```

> **`--dry-run=1` is the habit worth building.** Before running `setup:upgrade` on anything resembling production, dry-run it and read the SQL. It takes ten seconds and it is the difference between "adds a column" and "drops a column containing two years of leads".

Re-running a data patch during development

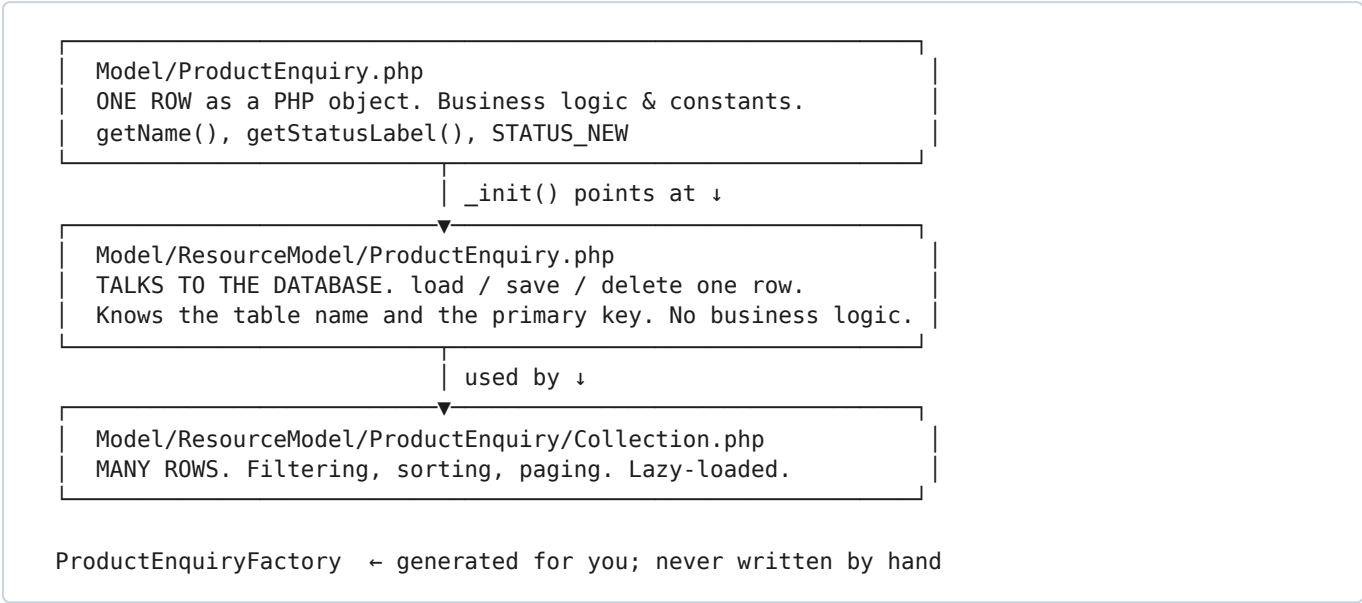
Data patches run once and are recorded in `patch_list`. To make `SeedProductEnquiries` run again:

```
DELETE FROM patch_list
WHERE patch_name = 'Codilar\\Twoxl\\Setup\\Patch\\Data\\SeedProductEnquiries';
```

Then `bin/magento setup:upgrade`. (Note the doubled backslashes in SQL.) Because your patch is idempotent — it checks `exists()` first — re-running it is harmless either way.

Part 9 — From a table to a working feature

A table alone does nothing. Magento's ORM needs four classes, and each has exactly one job.



9.1 The Model — one row

`app/code/Codilar/Twoxl/Model/ProductEnquiry.php`

```
class ProductEnquiry extends AbstractModel
{
    public const STATUS_NEW = 1;
    public const STATUS_IN_PROGRESS = 2;
    public const STATUS_CLOSED = 3;

    protected function _construct(): void
    {
        $this->_init(ProductEnquiryResource::class);
    }

    public function getStatusLabel(): string
    {
        return self::STATUSES[(int)$this->getStatus()] ?? '';
    }
}
```

Three things to understand:

- `_construct()` **with one underscore** is Magento's internal initialiser, *not* PHP's `__construct()`. It takes no arguments and its only job is `_init()`. Never put dependency injection here.
- **Getters and setters are magic.** `AbstractModel` extends `DataObject`, so `getEmail()` works without being written — it converts the method name to `email` and reads the data array. That is why your class carries `@method` annotations:

```
/**
 * @method string getName()
 * @method string getEmail()
 * @method int|null getStoreId()
 */
```

Those exist purely so your IDE and static analysis know the methods are real. They are documentation, not behaviour — but on a team project they are the difference between autocomplete working and not.

- **Business logic belongs here**, not in the resource model. `getStatusLabel()` is the right kind of method for this class.

9.2 The Resource Model — the database

`app/code/Codilar/Twoxl/Model/ResourceModel/ProductEnquiry.php`

```
class ProductEnquiry extends AbstractDb
{
    public const TABLE = 'codilar_twoxl_product_enquiry';

    protected function _construct(): void
    {
        $this->_init(self::TABLE, 'entity_id');
    }
}
```

Eleven lines, and that is the whole class. `_init(table, primaryKey)` is all `AbstractDb` needs to provide `load()`, `save()` and `delete()`.

Putting the table name in a `public const` is a small touch worth copying — anything that needs the raw table name (a direct SQL query, a join, a di.xml argument) can reference `ProductEnquiryResource::TABLE` instead of retyping the string.

> **Why model and resource model are separate.** This is the Data Mapper pattern. The model does not know SQL; the resource model does not know business rules. You can unit-test `getStatusLabel()` without a database, and you could swap the storage layer without touching business logic. It feels like ceremony on a small table, but it is the pattern every Magento entity follows — consistency is itself a feature.

9.3 The Collection — many rows

`app/code/Codilar/Twoxl/Model/ResourceModel/ProductEnquiry/Collection.php`

```
class Collection extends AbstractCollection
{
    protected $_idFieldName = 'entity_id';

    protected function _construct(): void
    {
        $this->_init(ProductEnquiry::class, ProductEnquiryResource::class);
    }

    /**
     * Enquiries still waiting on someone, newest first.
     */
    public function addOpenFilter(): self
    {
        $this->addFieldToFilter('status', ['neq' => ProductEnquiry::STATUS_CLOSED])
            ->setOrder('created_at', self::SORT_ORDER_DESC);

        return $this;
    }
}
```

Note `_init()` here takes **two** arguments: the model class and the resource class. The collection needs to know what to build (model) and where to read from (resource).

`addOpenFilter()` is the pattern to copy: a named method on the collection instead of the same two lines repeated in five controllers. It returns `$this`, so it chains:

```
$openEnquiries = $this->collectionFactory->create()->addOpenFilter()->setPageSize(10);
```

Collection filter syntax you will use constantly

```
->addFieldToFilter('status', 1)                            // = 1
->addFieldToFilter('status', ['neq' => 3])                 // != 3
->addFieldToFilter('status', ['in' => [1, 2]])             // IN (1,2)
->addFieldToFilter('email', ['like' => '%@example.com'])   // LIKE
->addFieldToFilter('created_at', ['gteq' => '2026-01-01']) // >=
->addFieldToFilter('sku', ['null' => true])                // IS NULL
->addFieldToFilter(['email', 'phone'], [                   // OR between fields
       ['like' => '%rahul%'], ['like' => '%9820%']
   ])
->setOrder('created_at', 'DESC')
->setPageSize(20)->setCurPage(1)
->getSize()      // COUNT(*), ignores paging
->getItems()     // runs the query, returns models
```

> **Collections are lazy.** No SQL runs until you call `getItems()`, `getSize()`, or iterate with `foreach`. That is why you can keep chaining filters. It is also why `$collection->getSize()` inside a loop is a performance bug — it re-runs a COUNT query each time.
>
> **Never call `->load()` yourself** on a collection. It is implicit, and calling it twice does not re-query — which leads to "my filter is being ignored" confusion.

9.4 The Factory — the class you never write

You inject `ProductEnquiryFactory` in six files, but no such file exists in your module. Magento **generates** it into `generated/code/Codilar/Twoxl/Model/ProductEnquiryFactory.php` the first time it is needed.

```
$enquiry = $this->enquiryFactory->create();   // a fresh, empty model
```

The rule is simple: **inject a factory for anything stateful** (models, which hold one row's data), and **inject the object directly** for anything stateless (resource models, config readers, services). Your code follows this exactly:

```
public function __construct(
    private readonly ProductEnquiryFactory $enquiryFactory,     // factory — stateful
    private readonly ProductEnquiryResource $enquiryResource,   // direct — stateless
    private readonly CollectionFactory $collectionFactory,      // factory — stateful
    private readonly StoreManagerInterface $storeManager        // direct — stateless
) {}
```

If you ever see "Class ...Factory does not exist", the fix is `bin/magento setup:di:compile`, or deleting `generated/` — never writing the file by hand.

9.5 Putting them together — the three operations

```
// CREATE
$enquiry = $this->enquiryFactory->create();
$enquiry->addData(['name' => 'Rahul', 'email' => 'r@example.com', 'status' => 1]);
$this->enquiryResource->save($enquiry);
$newId = $enquiry->getId();      // populated by save()

// READ
$enquiry = $this->enquiryFactory->create();
$this->enquiryResource->load($enquiry, 42);
if (!$enquiry->getId()) { /* not found — load() does NOT throw */ }

// DELETE
$this->enquiryResource->delete($enquiry);
```

The trap in `load()`: it does not throw when the id does not exist — it hands back an empty model. Every one of your admin controllers checks for this, and that is why they are correct:

```
$this->enquiryResource->load($enquiry, $id);
if (!$enquiry->getId()) {
    $this->messageManager->addErrorMessage(__('This enquiry no longer exists.'));
    return $redirect->setPath('*/*/index');
}
```

Skip that check and a deleted record produces a blank form that silently creates a *new* row when saved.

Part 10 — The admin UI on top of the table

Nine files turn your table into a working admin screen. Here is the chain, in the order a request travels.

```
Admin clicks the menu item
      |  menu.xml  →  action="twoxl_admin/enquiry/index"
      ▼
routes.xml  →  route id "twoxl_admin" belongs to Codilar_Twoxl
      ▼
Controller/Adminhtml/Enquiry/Index.php   (checks ADMIN_RESOURCE against acl.xml)
      ▼
layout/twoxl_admin_enquiry_index.xml  →  <uiComponent name="codilar_twoxl_enquiry_listing"/>
      ▼
ui_component/codilar_twoxl_enquiry_listing.xml   (columns, filters, buttons)
      ▼
di.xml  →  virtualType Grid\Collection  →  SearchResult over the table
      ▼
MySQL
```

10.1 Routing — `routes.xml`

```
<router id="admin">
    <route id="twoxl_admin" frontName="twoxl_admin">
        <module name="Codilar_Twoxl"/>
    </route>
</router>
```

This maps the URL `admin/twoxl_admin/enquiry/edit` to `Controller/Adminhtml/Enquiry/Edit.php`. The pattern is `frontName / controller-folder / action-class`.

You have a second one in `etc/frontend/routes.xml` with `router id="standard"` and frontName `twoxl` — that is the storefront side, serving `twoxl/enquiry/save`. Admin and frontend routes are declared in different files and never share a router.

10.2 Permissions — `acl.xml`

```
<resource id="Codilar_Twoxl::enquiries" title="Enquiries" sortOrder="71">
    <resource id="Codilar_Twoxl::enquiry" title="Product Enquiries" sortOrder="10"/>
</resource>
```

Every admin controller declares which permission it needs:

```
class Index extends Action implements HttpGetActionInterface
{
    public const ADMIN_RESOURCE = 'Codilar_Twoxl::enquiry';
```

> **Forget `ADMIN_RESOURCE` and the constant defaults to `Magento_Backend::admin`** — meaning any logged-in admin user can reach your controller, regardless of their role. This is one of the most common security findings in Magento code reviews. All four of your admin controllers declare it.

Your ACL also registers the two config sections, which is what makes `<resource>Codilar_Twoxl::enquiry_config</resource>` in `system.xml` work. Without the ACL entry the config section is invisible to everyone, including the admin who installed it.

10.3 The menu — `menu.xml`

Your file's comment explains the structure better than the docs do:

```
Levels map onto the admin menu the way Magento renders them:
  level 0 -> the sidebar icon, level 1 -> the group heading in the flyout, level 2 -> the
links.
```

So your three levels produce: **Enquiries** in the sidebar → **Manage** and **Settings** as headings → **Product Enquiries**, **New Enquiry** and **Configuration** as clickable links. A node with an `action` is a link; a node without one is a heading.

Note that the Configuration link points at a core action with your section id appended:

```
action="adminhtml/system_config/edit/section/twoxl"
```

Each menu node also carries `resource=`, so a role without the permission simply does not see the item.

10.4 The grid data source — `di.xml`

This is the part that surprises people: the grid does **not** use your `Collection` class. It uses a **virtual type**.

```
<virtualType name="Codilar\Twoxl\Model\ResourceModel\ProductEnquiry\Grid\Collection"
             type="Magento\Framework\View\Element\UiComponent\DataProvider\SearchResult">
    <arguments>
        <argument name="mainTable"
xsi:type="string">codilar_twoxl_product_enquiry</argument>
        <argument name="resourceModel"
xsi:type="string">Codilar\Twoxl\Model\ResourceModel\ProductEnquiry</argument>
    </arguments>
</virtualType>

<type name="Magento\Framework\View\Element\UiComponent\DataProvider\CollectionFactory">
    <arguments>
        <argument name="collections" xsi:type="array">
            <item name="codilar_twoxl_enquiry_listing_data_source"
xsi:type="string">Codilar\Twoxl\Model\ResourceModel\ProductEnquiry\Grid\Collection</item>
        </argument>
    </arguments>
</type>
```

Read it in two steps:

1. **A virtual type is a configured instance of an existing class, with no PHP file.** Here it is Magento's `SearchResult` told which table and resource model to use. You get grid filtering, sorting and paging without writing a line of code.
2. **The second block registers it under a name** — and that name, `codilar_twoxl_enquiry_listing_data_source`, must match the `<dataSource name="...">` in your listing XML exactly. A typo here produces an empty grid with no error message, which is a genuinely frustrating hour to debug.

> **Why not reuse your own `Collection`?** Grids need `SearchResultInterface` (addFilter, setSearchCriteria, getAggregations) which `AbstractCollection` does not implement. Keeping them separate means your `Collection` stays clean for business use — like `addOpenFilter()` — while the grid gets its own generic machinery.

10.5 The grid — `codilar_twoxl_enquiry_listing.xml`

Each column maps to a database column by name. Three deserve attention.

A select column backed by your source model

```
<column name="status" component="Magento_Ui/js/grid/columns/select" sortOrder="60">
    <settings>
        <options class="Codilar\Twoxl\Model\Source\EnquiryStatus"/>
        <filter>select</filter>
        <dataType>select</dataType>
        <label translate="true">Status</label>
    </settings>
</column>
```

The integer `1` in the database renders as "New" and filters as a dropdown — all from the one `OptionSourceInterface` class. The same class is referenced again in the edit form. One definition, two consumers.

A date column

```
<column name="created_at" class="Magento\Ui\Component\Listing\Columns\Date"
        component="Magento_Ui/js/grid/columns/date" sortOrder="80">
    <settings>
        <filter>dateRange</filter>
        <dataType>date</dataType>
        <label translate="true">Received</label>
    </settings>
</column>
```

The `Date` column class converts the stored UTC timestamp into the admin user's timezone. Without it the grid shows raw UTC and confuses everyone in GST+4.

The actions column

`app/code/Codilar/Twoxl/Ui/Component/Listing/Column/EnquiryActions.php`

```
$item[$this->getData('name')] = [
    'edit' => [
        'href'  => $this->urlBuilder->getUrl(self::EDIT_PATH, ['entity_id' => $id]),
        'label' => __('Edit'),
    ],
    'delete' => [
        'href'    => $this->urlBuilder->getUrl(self::DELETE_PATH, ['entity_id' => $id]),
        'label'   => __('Delete'),
        'post'    => true,
        'confirm' => [
            'title'   => __('Delete enquiry #%1', $id),
            'message' => __('Are you sure you want to delete this enquiry?'),
        ],
    ],
];
```

`prepareDataSource()` runs once per page and injects the links into each row. Two details matter:

- **`'post' => true`** — the delete link submits a POST, not a GET. Your `Delete` controller implements `HttpPostActionInterface`, so a GET is rejected outright. This prevents a crawler, a browser prefetch or a pasted URL from deleting records.
- **`'confirm'`** — a modal before the POST fires.

10.6 The edit form — `codilar_twoxl_enquiry_form.xml` + DataProvider

The form's `DataProvider` is what fills the fields:

```
public function getData(): array
{
    if ($this->loadedData === null) {
        $this->loadedData = [];
        foreach ($this->collection->getItems() as $enquiry) {
            $this->loadedData[$enquiry->getId()] = $enquiry->getData();
        }
    }
    return $this->loadedData;
}
```

The return shape is specific: **keyed by entity id**. The UI form component looks up the id from the request and takes that entry. Return a flat array and the form renders empty.

Each field names a database column, so the round trip works automatically:

```
<field name="email" formElement="input" sortOrder="20">
    <settings>
        <dataType>text</dataType>
        <label translate="true">Email</label>
        <validation>
            <rule name="required-entry" xsi:type="boolean">true</rule>
            <rule name="validate-email" xsi:type="boolean">true</rule>
        </validation>
    </settings>
</field>
```

And a hidden `entity_id` field is what makes the form update rather than insert:

```
<field name="entity_id" formElement="hidden">
```

10.7 The admin Save controller — the security detail

`app/code/Codilar/Twoxl/Controller/Adminhtml/Enquiry/Save.php`

```
/** Editable in the admin; everything else is whatever the storefront captured. */
private const EDITABLE = ['name', 'email', 'phone', 'sku', 'message', 'status'];
...
$enquiry->addData(array_intersect_key($data, array_flip(self::EDITABLE)));
```

This is the pattern to internalise. Never pass `$this->getRequest()->getPostValue()` straight into `addData()`. A crafted POST could then set `store_id`, `created_at`, or any column you never put on the form — a mass-assignment vulnerability. Intersecting against an explicit allow-list means only the six intended fields can ever be written, no matter what arrives.

Note what is deliberately **not** editable: `store_id` (a fact about where the enquiry came from), `created_at` and `updated_at` (owned by MySQL), and `entity_id` (handled separately as the load key).

10.8 The form buttons

Four small classes in `Block/Adminhtml/Enquiry/Edit/Button/`, each returning an array from `getButtonData()`. Two are worth reading closely.

Delete hides itself on a new record:

```
$id = $this->getEnquiryId();

if (!$id) {
    return [];          // empty array = button not rendered
}
```

Save and Continue passes a flag the controller reads:

```
// true keeps the form open; "back" is what Save::execute() looks for.
'params' => [true, ['back' => 'continue']],
```

and at the other end:

```
if ($this->getRequest()->getParam('back')) {
    return $redirect->setPath('*/*/edit', ['entity_id' => $enquiry->getId()]);
}
return $redirect->setPath('*/*/index');
```

That is the entire difference between the two save buttons: one parameter, read in one `if`.

Part 11 — The storefront write path

How a row actually comes into existence, end to end.

```
enquiry.phtml  (product page)
     │  POST to twoxl/enquiry/save, with form key
     ▼
Controller/Enquiry/Save.php
     │  1. is the feature enabled for THIS store view?
     │  2. is the form key valid?           (CSRF)
     │  3. validate & truncate the input
     │  4. stamp status + store_id
     ▼
ProductEnquiryResource::save()   →   INSERT INTO codilar_twoxl_product_enquiry
     │
     ▼
EnquiryNotifier::notify()  →  email, in that store's context, failures logged only
```

The four guards, in order, from `app/code/Codilar/Twoxl/Controller/Enquiry/Save.php`:

```
if (!$this->config->isEnabled()) { ... }                        // config, per store view
if (!$this->formKeyValidator->validate($this->request)) { ... }  // CSRF
$data = $this->validatedData();                                  // throws
LocalizedException
$enquiry->addData($data + [
    'status'   => ProductEnquiry::STATUS_NEW,
    'store_id' => (int)$this->storeManager->getStore()->getId(),
]);
```

The class comment states the principle that makes this correct:

> The form can be switched off, the phone field made mandatory and the message length capped from Stores > Configuration > Enquiries; all three are enforced here as well as in the template, because a post can arrive without the form having been rendered.

> **Client-side validation is a convenience; server-side validation is the actual rule.** The template sets `maxlength` and `required` attributes, and `validate-email` runs in the browser. None of that stops a POST sent with curl. Every one of those rules is re-checked in the controller — which is why `validatedData()` exists and why it uses `filter_var($email, FILTER_VALIDATE_EMAIL)` rather than trusting the form.

And after the row is saved, the notification is deliberately best-effort:

```
try {
    ...
    $transport->sendMessage();
} catch (\Throwable $e) {
    $this->logger->error('Could not send the product enquiry notification.', ['exception'
=> $e]);
}
```

The enquiry is already in the database by this point. A dead SMTP server must not show the shopper an error or lose their message. Save first, notify second, swallow and log — the right order for any side effect.

Part 12 — Five real traps in your own code

Trap 1 — The form that spun forever

From `codilar_twoxl_enquiry_form.xml`:

```
<!--
    Without this the form falls back to form/default.phtml, whose loading mask waits on
    "<name>.areas" - a component only a tabbed form creates. A single-fieldset form never
    registers it, so the mask never lifts and the page spins forever.
-->
<item name="template" xsi:type="string">templates/form/collapsible</item>
```

A one-line fix for a symptom that looks like a JavaScript error but is not. Any single-fieldset UI form needs this; every tabbed core form hides the problem.

Trap 2 — The grid export that printed raw numbers

Two separate fixes were needed to make CSV export readable.

The Store View column. It renders HTML, so the exporter cannot read a label out of it. Adding a `filterSelect` gives the exporter an options list to work from:

```
<filterSelect name="store_id" provider="${ $.parentName }">
    <settings>
        <options class="Magento\Store\Ui\Component\Listing\Column\Store\Options"/>
```

The date column. Without registering it with the export metadata provider in `di.xml`, "Received" exports as raw UTC rather than the admin's timezone:

```
<!-- Exports "Received" in the admin's timezone, the way the grid shows it, not raw UTC. -
->
<type name="Magento\Ui\Model\Export\MetadataProvider">
    <arguments>
        <argument name="data" xsi:type="array">
            <item name="codilar_twoxl_enquiry_listing" xsi:type="array">
                <item name="created_at" xsi:type="string">created_at</item>
```

Lesson: **what the grid shows and what the grid exports are produced by different code paths.** Always test the export, not just the screen.

Trap 3 — Export needs the selections column

The comment in the listing says it:

```
CSV and Excel XML come from the exportButton definition. It reads the current
filters and ticked rows off the "ids" selections column below, so the two go together.
```

`<exportButton>` without `<selectionsColumn name="ids">` produces an export button that does nothing useful. They are a pair.

Trap 4 — Delete must be POST-only

The `Delete` controller implements `HttpPostActionInterface`, so the button cannot be an ordinary link:

```
// deleteConfirm() posts the request, which the POST-only Delete controller requires.
'on_click' => sprintf("deleteConfirm('%s', '%s', {});", $message->render(), $this->getUrl(...)),
```

Use `location.href` here instead and the delete silently fails with a 404-ish error. The grid's actions column solves the same problem with `'post' => true`.

Trap 5 — Two `Save` classes with the same name

Your module has `Controller/Enquiry/Save.php` (storefront) and `Controller/Adminhtml/Enquiry/Save.php` (admin). Different namespaces, completely different jobs, and very different security postures:

	Storefront Save	Admin Save
Who can call it	Anyone, including guests	Logged-in admin with the ACL resource
CSRF protection	Form key validator, explicitly	Automatic for admin routes
Validation	Full — email format, required fields, length caps	Allow-list of editable fields
Sets `store_id`	Yes, from the current store view	No — never editable
On failure	Message + redirect to referer	Message + redirect to edit form

When you open one of these files, check the namespace first. Editing the wrong one is an easy mistake and the consequences are asymmetric — a missing check on the storefront side is public.

Part 13 — Cheat sheet, commands and exercises

The complete file list for one custom table

File	Purpose	Required?
`etc/db_schema.xml`	The table structure	Yes
`etc/db_schema_whitelist.json`	Proof of ownership for safe drops	Yes
`Model/ProductEnquiry.php`	One row + business logic	Yes
`Model/ResourceModel/ProductEnquiry.php`	load / save / delete	Yes
`Model/ResourceModel/ProductEnquiry/Collection.php`	Many rows, filtered	Yes
`etc/di.xml`	Grid collection virtual type	For a grid
`etc/acl.xml`	Permissions	For admin
`etc/adminhtml/menu.xml`	Sidebar entry	For admin
`etc/adminhtml/routes.xml`	URL → controller	For admin
`Controller/Adminhtml/<Entity>/{Index,Edit,Save,Delete}.php`	The four admin actions	For admin
`view/adminhtml/layout/*_index.xml` / `*_edit.xml`	Place the UI component	For admin
`view/adminhtml/ui_component/*_listing.xml`	The grid	For a grid
`view/adminhtml/ui_component/*_form.xml`	The edit form	For a form
`Model/<Entity>/DataProvider.php`	Feeds the form	For a form
`Ui/Component/Listing/Column/*Actions.php`	Edit/Delete links	For a grid
`Block/Adminhtml/<Entity>/Edit/Button/*.php`	Form buttons	For a form
`Model/Source/*.php`	Dropdown options from constants	If you have an enum
`Setup/Patch/Data/Seed*.php`	Demo/initial rows	Optional

A skeleton table to copy

```
<?xml version="1.0"?>
<schema xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

xsi:noNamespaceSchemaLocation="urn:magento:framework:Setup/Declaration/Schema/etc/schema.x
sd">
    <table name="vendor_module_entity" resource="default" engine="innodb"
           comment="What this table is for">

        <column xsi:type="int" name="entity_id" unsigned="true" nullable="false"
                identity="true" comment="Entity ID"/>
        <column xsi:type="varchar" name="title" length="255" nullable="false"
comment="Title"/>
        <column xsi:type="text" name="body" nullable="true" comment="Body"/>
        <column xsi:type="smallint" name="status" unsigned="true" nullable="false"
                default="1" comment="1 enabled, 2 disabled"/>
        <column xsi:type="decimal" name="amount" precision="20" scale="4" nullable="true"
                default="0" comment="Amount"/>
        <column xsi:type="smallint" name="store_id" unsigned="true" nullable="true"
                comment="Store view"/>
        <column xsi:type="timestamp" name="created_at" on_update="false" nullable="false"
                default="CURRENT_TIMESTAMP" comment="Created at"/>
        <column xsi:type="timestamp" name="updated_at" on_update="true" nullable="false"
                default="CURRENT_TIMESTAMP" comment="Updated at"/>

        <constraint xsi:type="primary" referenceId="PRIMARY">
            <column name="entity_id"/>
        </constraint>
        <constraint xsi:type="foreign"
referenceId="VENDOR_MODULE_ENTITY_STORE_ID_STORE_STORE_ID"
                    table="vendor_module_entity" column="store_id"
                    referenceTable="store" referenceColumn="store_id" onDelete="SET
NULL"/>

        <index referenceId="VENDOR_MODULE_ENTITY_STATUS" indexType="btree">
            <column name="status"/>
        </index>
    </table>
</schema>
```

Commands

```
# the loop you will run a hundred times
bin/magento setup:db-declaration:generate-whitelist --module-name=Codilar_Twoxl
bin/magento setup:upgrade
bin/magento cache:flush

# safety first on anything shared
bin/magento setup:upgrade --dry-run=1
bin/magento setup:db:status

# when a Factory or Interceptor "does not exist"
bin/magento setup:di:compile
# or, in developer mode, just:  rm -rf generated/

# admin UI not updating after an XML change
bin/magento cache:clean config full_page layout
```

SQL for checking your work

```
-- the structure Magento actually built
SHOW CREATE TABLE codilar_twoxl_product_enquiry\G

-- columns, types, nullability and defaults
DESCRIBE codilar_twoxl_product_enquiry;

-- the indexes that exist (note store_id appears via the FK)
SHOW INDEX FROM codilar_twoxl_product_enquiry;

-- the foreign keys
SELECT constraint_name, column_name, referenced_table_name, referenced_column_name
FROM   information_schema.key_column_usage
WHERE  table_name = 'codilar_twoxl_product_enquiry'
       AND referenced_table_name IS NOT NULL;

-- the data, with the status spelled out and the store named
SELECT e.entity_id, e.name, e.email, e.sku,
       CASE e.status WHEN 1 THEN 'New' WHEN 2 THEN 'In Progress' ELSE 'Closed' END AS
status,
       s.code AS store_view, e.created_at
FROM   codilar_twoxl_product_enquiry e
LEFT   JOIN store s ON s.store_id = e.store_id
ORDER  BY e.created_at DESC;

-- did the data patch register?
SELECT * FROM patch_list WHERE patch_name LIKE '%Twoxl%';

-- prove an index is being used
EXPLAIN SELECT * FROM codilar_twoxl_product_enquiry WHERE status != 3 ORDER BY created_at
DESC;
```

Exercises

1. **Read the real structure.** Run `SHOW CREATE TABLE` and match every line back to your `db_schema.xml`. Find the index on `store_id` that you never declared, and explain where it came from.
2. **Add a column properly.** Add `<column xsi:type="varchar" name="assigned_to" length="120" nullable="true"/>`. Regenerate the whitelist, diff the JSON, dry-run the upgrade and read the SQL before applying it. Then surface it in the grid and the form.
3. **Break the whitelist on purpose.** Add a column, run `setup:upgrade` *without* regenerating the whitelist, then remove it from the XML and upgrade again. Confirm with `DESCRIBE` that the column is still in the database. Now regenerate the whitelist and upgrade — watch it finally drop.
4. **Prove the foreign key works.** In SQL, try `UPDATE codilar_twoxl_product_enquiry SET store_id = 99 WHERE entity_id = 1;`. It will be rejected — store 99 does not exist. That rejection is the constraint doing its job.
5. **Prove `SET NULL` works.** On a throwaway install, create a test store view, add an enquiry in it, delete the store view, then look at the row. The enquiry survives with `store_id = NULL`. Imagine the same test with `CASCADE`.
6. **Watch `updated_at` move by itself.** Note the value, edit the enquiry in the admin, save, and look again. Then grep your module for `updated_at` — you will find it only in the schema. MySQL did it.
7. **Use the collection.** Write a tiny script or a temporary block that calls `$collectionFactory->create()->addOpenFilter()->setPageSize(5)` and prints the results. Then add `->getSelect()->__toString()` to see the SQL it generated.
8. **Test the export.** Export the grid to CSV. Confirm Store View shows names and Received shows local time — then comment out the `MetadataProvider` block in `di.xml`, flush cache, and export again to see the difference.
9. **Break mass-assignment protection.** Temporarily replace `array_intersect_key(...)` in the admin Save controller with `$data`, then POST a `store_id` the form never rendered. Watch it get written. Restore the allow-list immediately and remember why it is there.
10. **Build a second table from scratch.** `codilar_twoxl_enquiry_note` — `note_id`, `enquiry_id` (FK to your table, `CASCADE` this time, because a note is meaningless without its enquiry), `author`, `body`, `created_at`. Model, resource, collection, and a grid. If you can do that without looking anything up, you have learned this topic.

Checklist for every new table

1. Name it `vendor_module_entity`, lowercase, under 40 characters.
2. `engine="innodb"`, `resource="default"`, and a table `comment`.
3. An `identity="true"` primary key.
4. Every column gets a `comment` and an explicit `nullable`.
5. `decimal` for money. Never `float`.
6. Integer + constants + a Source model for anything enum-like.
7. `created_at` / `updated_at` with the `on_update` pair.
8. Foreign key column types must match the referenced column **exactly**.
9. Choose `onDelete` deliberately — `SET NULL` for context, `CASCADE` only for true children.

10. Index what you filter, sort or join on. Nothing else. Skip FK columns — they are indexed already.
11. Regenerate the whitelist and commit both files together.
12. Dry-run before upgrading anything shared.
13. Validate and truncate in PHP to match the column lengths, with `mb_substr`.
14. Allow-list editable fields in every save controller.
15. Declare `ADMIN_RESOURCE` on every admin controller.
16. Make destructive actions POST-only.

> **The one-sentence summary:** `db_schema.xml` describes what the table **should look like**; Magento works out how to get there; the whitelist is how it knows what it is allowed to remove; and the Model / ResourceModel / Collection trio is how PHP talks to it.